

Juan Carrasco Milla

9 de junio de 2025

Índice

Documentacion_TFG	1
1. Requisitos previos	3
2. Iniciar el proceso con Certbot	3
3. Gestionar el registro TXT en DuckDNS	3
3.1. Borrar el registro TXT anterior (opcional pero recomendable):	3
3.2. Crear el nuevo registro TXT con el valor que te da Certbot:	4
4. Verificar la propagación del registro TXT	4
5. Completar el proceso en Certbot	4
6. Configurar el servidor web	4
Ejemplo Nginx:	4
Ejemplo Apache:	4
7. (Opcional) Limpiar el registro TXT	5
8. Renovación del certificado	5
9. Notas y consejos	5
10. Referencias	5

Documentacion_TFG

Resumen

VirtuStack desarrolla una solución integral para la gestión y sincronización automatizada de máquinas virtuales y usuarios en un entorno de infraestructura IT, utilizando un dashboard web propio y la plataforma de automatización AWX. La aplicación, construida con tecnologías modernas como Next.js, React, Node.js y MySQL, permite a los administradores crear, monitorizar y sincronizar recursos de forma segura y eficiente, integrando autenticación, ejecución remota de comandos con asistente de IA y gestión centralizada de credenciales y usuarios.

Contextualización del Proyecto

En entornos IT modernos, la gestión eficiente de recursos virtualizados y usuarios es fundamental para garantizar la seguridad, la escalabilidad y la operatividad de los servicios. Plataformas como AWX (la versión open-source de Ansible Tower) facilitan la automatización de tareas, pero muchas organizaciones requieren interfaces personalizadas que se adapten a sus flujos de trabajo y sistemas existentes. Este proyecto surge para cubrir esa necesidad, proporcionando una capa de integración entre AWX y una aplicación web propia que centraliza la administración de máquinas virtuales y usuarios.

Justificación de la Elección del Tema

La automatización y la administración centralizada de infraestructuras virtuales son tendencias clave en la evolución de los sistemas IT. Sin embargo, la integración entre herramientas de automatización como AWX y aplicaciones personalizadas no siempre es trivial, especialmente cuando se requiere una gestión segura de usuarios, contraseñas y recursos virtuales. Elegir este tema permite abordar un reto real en la industria, aplicando tecnologías actuales y buenas prácticas de desarrollo seguro y escalable.

Objetivos del Proyecto

- Objetivo General

Desarrollar una plataforma web que integre la gestión de máquinas virtuales y su administración mediante IA y usuarios con AWX, permitiendo la automatización y sincronización de recursos de forma segura y eficiente.

- Objetivos Específicos

Implementar un dashboard web intuitivo para la creación y gestión de máquinas virtuales y usuarios.

Integrar la aplicación con AWX mediante su API REST, automatizando la creación y asociación de usuarios y organizaciones.

Garantizar la seguridad en el almacenamiento y transmisión de credenciales, empleando hash bcrypt para contraseñas.

Ejecutar comandos remotos en las VMs mediante SSH y reflejar los resultados en tiempo real con la asistencia de Gemini

Sincronizar el estado de las VMs y usuarios entre la base de datos local y AWX.

Proporcionar un sistema robusto de manejo de errores y validación tanto en frontend como en backend.

Marco Teórico

- 1 Virtualización: Tecnología que permite ejecutar múltiples sistemas operativos en una sola máquina física, optimizando recursos y facilitando la administración. Proximos será el sistema de virtualización para el desarrollo de este proyecto
- 2
- 3 Automatización IT: Uso de herramientas como Ansible/AWX para la orquestación y automatización de tareas de infraestructura, mejorando la eficiencia y reduciendo errores humanos.

```
4
5 AWX: Plataforma open-source que proporciona una interfaz web y API REST para
  gestionar y automatizar tareas con Ansible.
6
7 Node.js y Next.js: Entorno y framework para el desarrollo de aplicaciones web,
  permitiendo la integración de backend y frontend en un mismo proyecto.
8
9 MySQL: Sistema de gestión de bases de datos relacional, empleado para almacenar
  información de usuarios, máquinas virtuales y credenciales.
10
11 SSH: Protocolo seguro para la administración remota de sistemas, utilizado aquí para
  ejecutar comandos en las VMs.
12
13 Seguridad de contraseñas: Uso de algoritmos de hash como bcrypt para almacenar
  contraseñas de forma segura, evitando el almacenamiento de texto plano.
```

Metodología

```
1  Análisis de requisitos: Identificación de necesidades de gestión y sincronización
  entre la aplicación y AWX.
2
3  Diseño de arquitectura: Definición de la estructura de la base de datos, endpoints de
  la API y flujos de integración con AWX.
4
5  Desarrollo incremental: Implementación iterativa de funcionalidades, comenzando por
  la gestión básica de VMs y usuarios, y añadiendo integración con AWX y SSH.
6
7  Pruebas y validación: Verificación funcional de los endpoints, pruebas de seguridad (
  hash de contraseñas, manejo de errores) y validación de la integración con AWX.
```

Infraestructura

Imagen

Proxy mkdir -p /etc/nginx/sites-available/ 63 systemctl status nginx 64 apt install nginx 65 vim /etc/nginx/sites-available/cromopolis.duckdns.org 66 sudo ln -s /etc/nginx/sites-available/cromopolis.duckdns.org /etc/nginx/sites-enabled/ 67 sudo nginx -t 68 sudo systemctl reload nginx

```
1  server {
2      listen 80;
3      server_name cromopolis.duckdns.org;
4      return 301 https://$host$request_uri;
5  }
6
7  server {
8      listen 443 ssl;
9      server_name cromopolis.duckdns.org;
10
11     ssl_certificate /etc/letsencrypt/live/cromopolis.duckdns.org/fullchain.pem;
12     ssl_certificate_key /etc/letsencrypt/live/cromopolis.duckdns.org/privkey.pem;
13
14     ssl_protocols TLSv1.2 TLSv1.3;
15     ssl_ciphers HIGH:!aNULL:!MD5;
16
17     # Proxy para WebSocket SSH interactivo
18     location /ws/ {
19         proxy_pass http://192.168.0.194:8080/;
20         proxy_http_version 1.1;
21         proxy_set_header Upgrade $http_upgrade;
22         proxy_set_header Connection "Upgrade";
23         proxy_set_header Host $host;
24         proxy_read_timeout 3600;
25     }
26 }
```

```

27     # Proxy para tu app Next.js
28     location / {
29         proxy_pass http://192.168.0.194:3000;
30         proxy_set_header Host $host;
31         proxy_set_header X-Real-IP $remote_addr;
32         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
33         proxy_set_header X-Forwarded-Proto $scheme;
34     }
35 }

```

Certificado SSL

Certificado https: # Documentación: Obtención de Certificado SSL Let's Encrypt para DuckDNS (Desafío DNS Manual)

1. Requisitos previos

- *Dominio DuckDNS* activo (ejemplo: cromopolis.duckdns.org)
- *Token DuckDNS* (lo obtienes en <https://www.duckdns.org/>)
- *Servidor Linux* con acceso a internet
- *Certbot* instalado (sudo apt install certbot)

2. Iniciar el proceso con Certbot

Ejecuta el siguiente comando para iniciar la solicitud del certificado:

```
sudo certbot certonly --manual --preferred-challenges dns -d cromopolis.duckdns.org
```

text

Certbot mostrará un mensaje parecido a este:

Please deploy a DNS TXT record under the name:

_acme-challenge.cromopolis.duckdns.org.

with the following value:

VALOR_UNICO_QUE_TE_DA_CERTBOT

text

3. Gestionar el registro TXT en DuckDNS

3.1. Borrar el registro TXT anterior (opcional pero recomendable):

```
curl "https://www.duckdns.org/update?domains=cromopolis&token=TU_TOKEN&clear=true"
```

text

3.2. Crear el nuevo registro TXT con el valor que te da Certbot:

```
curl "https://www.duckdns.org/update?domains=cromopolis&token=TU_TOKEN&txt=VALOR_UNICO_QUE_TE_DA_CERTBOT"
```

text

- Sustituye TU_TOKEN por tu token real de DuckDNS.
 - Sustituye VALOR_UNICO_QUE_TE_DA_CERTBOT por el valor exacto que te muestra Certbot.
-

4. Verificar la propagación del registro TXT

Antes de continuar, verifica que el registro TXT esté visible en servidores DNS públicos:

```
dig _acme-challenge.cromopolis.duckdns.org TXT @8.8.8.8 dig _acme-challenge.cromopolis.duckdns.org TXT @1.1.1.1  
dig _acme-challenge.cromopolis.duckdns.org TXT @9.9.9.9
```

text

Debes ver solo el valor nuevo que te dio Certbot en la respuesta.

5. Completar el proceso en Certbot

Cuando el registro TXT esté correctamente propagado y visible en los DNS públicos, vuelve a la terminal donde Certbot espera y pulsa *Enter*.

Si todo está correcto, Certbot mostrará un mensaje de éxito y los archivos generados:

- Certificado: /etc/letsencrypt/live/cromopolis.duckdns.org/fullchain.pem
 - Clave privada: /etc/letsencrypt/live/cromopolis.duckdns.org/privkey.pem
-

6. Configurar el servidor web

Ejemplo Nginx:

```
ssl_certificate /etc/letsencrypt/live/cromopolis.duckdns.org/fullchain.pem; ssl_certificate_key /etc/letsencrypt/live/cromopolis.duckdns.org/privkey.pem;
```

text

Ejemplo Apache:

```
SSLCertificateFile /etc/letsencrypt/live/cromopolis.duckdns.org/fullchain.pem SSLCertificateKeyFile /etc/letsencrypt/live/cromopolis.duckdns.org/privkey.pem
```

text

7. (Opcional) Limpiar el registro TXT

Cuando termines, puedes limpiar el registro TXT de DuckDNS:

```
curl "https://www.duckdns.org/update?domains=cromopolis&token=TU_TOKEN&clear=true"
```

text

8. Renovación del certificado

¡IMPORTANTE!

El método manual *NO se renueva automáticamente*.

Debes repetir este proceso antes de que el certificado expire (cada 90 días).

9. Notas y consejos

- Siempre usa el valor TXT más reciente que te da Certbot.
 - No pulses Enter en Certbot hasta que el registro TXT esté propagado y visible en DNS públicos.
 - Si tienes problemas de propagación, espera 1-2 minutos y vuelve a verificar.
 - Puedes automatizar el proceso usando el plugin oficial de DuckDNS para Certbot.
-

10. Referencias

- [Let's Encrypt](#)
 - [DuckDNS](#)
 - [Certbot](#)
 - [Herramienta online para consultar registros TXT](#)
-

Portainer

<https://192.168.0.201:9443/#!/2/docker/containers>

Imágenes

```
1 services:
2   duckdns:
3     image: lscr.io/linuxserver/duckdns:latest
4     container_name: duckdns
5     network_mode: host #optional
6     environment:
7       - PUID=1000 #optional
8       - PGID=1000 #optional
9       - TZ=Etc/UTC #optional
10      - SUBDOMAINS=cromopolis
```

```

11     - TOKEN=cea1bdf0-d166-4061-b51a-1920e51ba9fe
12     - UPDATE_IP=ipv4 #optional
13     - LOG_FILE=false #optional
14     volumes:
15     - /root/docker/duckdns/config:/config #optional
16     restart: unless-stopped

```

Wireguard

<http://192.168.0.202:10086/#/>

Captura

```

root@wireguard:~# cat /etc/wireguard/wg0.conf [Interface] Address = 10.0.0.1/24 SaveConfig = true PostUp = iptables -A
FORWARD -i wg0 -j ACCEPT; iptables -A FORWARD -o wg0 -j ACCEPT; iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE;
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT; iptables -D FORWARD -o wg0 -j ACCEPT; iptables -t nat -D POSTROUTING
-o eth0 -j MASQUERADE; ListenPort = 51820 PrivateKey = GJ+ANSThhqGe2igeHMc69uNACAPN2ZzBkBNWfii35GI=

```

```

[Peer] PublicKey = mWGWa9ZcuiTanbPtv/EQxkbo9cpg2nt5ZHkAPxMIDk8= AllowedIPs = 10.0.0.2/32 Endpoint =
78.136.121.201:51820

```

```

[Peer] PublicKey = wR7oUshRReAHWgCFckzH/GLZysC5ypHu877083XXz2I= AllowedIPs = 10.0.0.3/32

```

```

[Peer] PublicKey = wZ5NPbt0X6Lglzg1spIDKt9aPZ3hXu3RQZ9VD9rnETQ= AllowedIPs = 10.0.0.4/32 root@wireguard:~#

```

DDNS

Duck DNS se utiliza como servicio de DNS dinámico (DDNS) para asociar un subdominio a la IP pública de nuestra red, lo que resulta esencial cuando no disponemos de una IP fija. En este proyecto, se emplea un contenedor de Docker que actualiza automáticamente la IP pública registrada en Duck DNS cada pocos minutos, asegurando que el subdominio apunte siempre a la dirección correcta incluso si la IP cambia. De este modo, es posible acceder de forma remota y sencilla a los servicios desplegados en la red local, sin preocuparse por los cambios de dirección IP asignados por el proveedor de Internet

Cloud init

Cloud-init es una herramienta ampliamente utilizada para automatizar la configuración inicial de máquinas virtuales Linux en entornos de nube o virtualización. Permite personalizar una instancia durante su primer arranque (o en arranques posteriores) aplicando configuraciones como la creación de usuarios, la instalación de paquetes, la configuración de redes, la asignación de claves SSH, o la ejecución de scripts personalizados

Gitlab

GitLab es una plataforma de gestión de código fuente y control de versiones basada en Git, ampliamente utilizada para almacenar, organizar y colaborar en proyectos de software. En este proyecto, GitLab se emplea como repositorio centralizado donde se almacena todo el código fuente del dashboard, el backend y los scripts de integración con AWX, lo que facilita el trabajo colaborativo y el seguimiento de los cambios realizados a lo largo del desarrollo

El uso de GitLab permite aprovechar el control de versiones distribuido, de modo que cada desarrollador puede trabajar con una copia local del proyecto, crear ramas para nuevas funcionalidades o correcciones, y fusionar los cambios de forma segura y transparente en la rama principal

Plataforma

1		
2	components.json	
3	eslint.config.mjs	
4	hash-password.ts	

5	next.config.ts		—
6	next-env.d.ts		—
7	package.json		—
8	package-lock.json		—
9	postcss.config.mjs		—
10	public		
11	file.svg		—
12	globe.svg		—
13	next.svg		—
14	vercel.svg		—
15	window.svg		—
16	README.md		—
17	src		
18	app		
19	api		
20	auth		
21	[...nextauth]		
22	route.ts		
23	create-vm		
24	route.ts		
25	login		
26	route.ts		
27	logout		
28	route.ts		
29	me		
30	route.ts		
31	register		
32	route.ts		
33	sync-awx		
34	route.ts		
35	vms		
36	route.ts		
37	[vmid]		
38	exec		
39	route.ts		
40	metrics		
41	route.ts		
42	mysql-action		
43	route.ts		
44	reboot		
45	route.ts		
46	route.ts		
47	start		
48	route.ts		
49	stop		
50	route.ts		
51	suggest		
52	route.ts		
53	create-vm		
54	page.tsx		
55	dashboard		
56	page.tsx		
57	favicon.ico		
58	globals.css		
59	layout.tsx		

60		login
61		page.tsx
62		manage-vms
63		page.tsx
64		[vmid]
65		admin
66		Console.tsx
67		page.tsx
68		page.tsx
69		suggest
70		page.tsx
71		page.tsx
72		register
73		page.tsx
74		components
75		AutomationsSection.tsx
76		CTA.tsx
77		Features.tsx
78		Footer.tsx
79		Hero.tsx
80		Navbar.tsx
81		TechStack.tsx
82		ui
83		badge.tsx
84		button.tsx
85		card.tsx
86		tabs.tsx
87		textarea.tsx
88		VmCpuChart.tsx
89		VMList.tsx
90		lib
91		db.ts
92		proxmox.ts
93		utils.ts
94		middleware.ts
95		ssh-ws-server.js
96		types
97		nextauth.d.ts
98		tsconfig.json

Este proyecto es una plataforma web para la gestión y automatización de máquinas virtuales y usuarios, integrando servicios como AWX y Proxmox. Está desarrollado con Next.js (React) y Node.js, y utiliza una arquitectura modular con separación clara entre frontend, backend (API), componentes y utilidades.

1	Frontend:
2	Las páginas de usuario, como el dashboard, login, registro y la creación y gestión de VMs, se encuentran en src/app/. Estas páginas interactúan con la API mediante formularios y peticiones HTTP.
3	
4	Backend/API:
5	La lógica de negocio y las integraciones externas residen en los endpoints de la API, ubicados en src/app/api/. Aquí se gestionan operaciones como autenticación, registro de usuarios, creación de VMs, sincronización con AWX, y acciones sobre las máquinas virtuales (iniciar, detener, reiniciar, obtener métricas, ejecutar comandos, etc.).
6	
7	Componentes reutilizables:

8 En `src/components/` se encuentran los elementos de interfaz reutilizables y secciones del dashboard, como tablas de VMs, gráficos de uso de CPU, formularios y elementos UI personalizados.

9

10 Integración y utilidades:

11 El directorio `src/lib/` contiene la lógica de conexión con la base de datos (`db.ts`), integración con Proxmox (`proxmox.ts`), y funciones auxiliares (`utils.ts`).

12

13 Gestión de autenticación:

14 Se utiliza NextAuth para la autenticación de usuarios, con rutas protegidas y middleware (`middleware.ts`) para asegurar el acceso.

15

16 Scripts y configuración:

17 Archivos como `hash-password.ts` permiten operaciones específicas (por ejemplo, generar hashes bcrypt para contraseñas). Los archivos de configuración (`next.config.ts`, `tsconfig.json`, etc.) definen el entorno de desarrollo y despliegue.

18

19 Recursos estáticos:

20 Imágenes y archivos públicos se almacenan en la carpeta **public/**.

Resultados

- 1 Se ha conseguido una integración completa y funcional entre el dashboard propio y AWX , permitiendo la gestión centralizada de máquinas virtuales y usuarios.
- 2
- 3 El sistema garantiza la seguridad de las credenciales mediante el uso de hash bcrypt y la transmisión segura de datos.
- 4
- 5 La automatización de tareas administrativas y la sincronización de datos han reducido el esfuerzo manual y el riesgo de errores humanos.
- 6
- 7 El dashboard ofrece una experiencia de usuario intuitiva y robusta, con validación de datos y mensajes de error claros.
- 8
- 9 Los objetivos generales y específicos han sido alcanzados, demostrando la viabilidad y utilidad de la solución propuesta.